Projet sur la programmation concourante en Java

Titre : Simulation d'un pont mobile bidirectionnel pour la gestion synchronisée des véhicules et des bateaux à l'aide de threads en Java

1. Contexte général

On considère un pont mobile qui relie deux côtés d'une route et permet également le passage de bateaux circulant sous le pont. Le pont constitue une ressource partagée utilisée par plusieurs catégories de véhicules, dont les besoins en espace et les priorités de passage sont différents.

Le système doit gérer simultanément quatre flux de circulation :

- Les véhicules circulant de gauche à droite.
- Les véhicules circulant de droite à gauche.
- Les bateaux circulant du nord vers le sud.
- Les bateaux circulant du sud vers le nord.

Chaque véhicule et chaque bateau est représenté par un thread Java indépendant. La simulation doit coordonner ces threads afin d'éviter les conflits d'accès aux ressources, de respecter les règles de circulation et de garantir la sécurité du pont.

L'objectif principal est de mettre en œuvre les mécanismes de programmation concurrente permettant de gérer les ressources partagées, les files d'attente, les priorités et les différentes contraintes de circulation.

2. Gestion des états du pont mobile

Le pont possède quatre états : (on peut éventuellement avoir deux états: FERMÉ et OUVERT)

- FERMÉ : le pont est fermé à la circulation des bateaux et peut accueillir les véhicules.
- OUVERTURE : le pont est en cours d'ouverture. Aucun nouveau véhicule terrestre ne peut entrer sur le pont.
- OUVERT : le pont est ouvert pour permettre le passage des bateaux.
- FERMETURE : le pont est en cours de fermeture. Aucun véhicule terrestre ne peut entrer avant le retour à l'état FERMÉ.

Les transitions autorisées sont : FERMÉ → OUVERTURE → OUVERT → FERMETURE → FERMÉ.

Le pont ne peut commencer à s'ouvrir que lorsque tous les véhicules qui l'occupent ont terminé leur traversée et l'ont complètement quitté.

Lorsqu'une demande d'ouverture est prise en charge, les entrées du pont sont bloquées pour les nouveaux véhicules. Les véhicules déjà engagés terminent leur traversée avant le début de l'ouverture.

Aucun véhicule ne peut traverser le pont pendant les états OUVERTURE, OUVERT ou FERMETURE.

3. Gestion des segments du pont

Le pont est divisé en N segments consécutifs : S1, S2, S3, ..., SN.

Chaque segment constitue une ressource partagée dont l'occupation doit être contrôlée par le programme.

Les véhicules circulant dans le sens opposé suivent le même principe, en parcourant leurs segments de l'autre sens. Donc plusieurs véhicules peuvent se trouver simultanément sur le pont, dans les deux sens, si les segments nécessaires sont disponibles. Aucun véhicule n'occupe les segments de l'autre sens (pas de dépassement, ni de demi-tour sur le pont).

3.1. Voitures

Une voiture occupe un seul segment à la fois. Pour traverser le pont, elle doit :

1. Attendre que le premier segment nécessaire soit libre.
2. Réserver ce segment.
3. Avancer vers le segment suivant lorsque celui-ci est disponible.
4. Libérer le segment précédent lorsque la voiture l'a effectivement quitté.
5. Répéter ce processus jusqu'à la sortie complète du pont.

3.2. Camions

Un camion occupe deux segments consécutifs en raison de sa longueur.

Avant d'avancer, il doit pouvoir réserver les deux segments nécessaires simultanément. Donc Il ne peut pas occuper un segment et attendre le second. Il progresse ensuite sur le pont en libérant les deux segments qu'il quitte et en réservant les suivants, jusqu'à sa sortie complète.

Les mêmes règles s'appliquent aux deux sens de circulation.

3.3. Convois exceptionnels

Un convoi exceptionnel occupe quatre segments consécutifs en raison de sa longueur. Sa largeur ne permet pas la présence simultanée de véhicules circulant dans le sens opposé, donc, le convoi traverse le pont sans présence simultanée de véhicules dans le sens opposé.

Lorsqu'un convoi exceptionnel arrive à l'entrée du pont :

1. L'entrée du pont dans le sens opposé est immédiatement bloquée pour les nouveaux véhicules, mais les véhicules déjà présents peuvent terminer leur traversée
2. Le convoi attend que tous les véhicules du sens opposé aient quitté le pont et que les quatre segments nécessaires soient disponibles pour entrer au pont.
3. Les véhicules circulant dans le même sens que le convoi, peuvent continuer à avancer normalement, dans la limite des segments disponibles.
4. Après sa sortie complète, l'entrée opposée est débloquée, sauf si une autre contrainte de priorité ou de sécurité impose son maintien fermé

4. Gestion des bateaux

Les bateaux circulent sous le pont, dans les deux directions : nord vers sud et sud vers nord. Un seul bateau peut passer à la fois sur l'espace occupé par le pont.

4.1. File d'attente FIFO

Tous les bateaux sont enregistrés dans une file d'attente FIFO commune (le premier bateau arrivé est le premier à passer), indépendamment de leur direction.

Lorsqu'un bateau arrive :

- Sa demande est enregistrée dans la file. Donc, Il attend son tour (si un autre bateau est arrivé plus tôt, ce dernier doit passer avant lui).

Cette règle s'applique aussi bien lorsque le pont est fermé que lorsqu'il est ouvert et que plusieurs bateaux attendent leur passage.

4.2. Ouverture et fermeture

Lorsqu'une ouverture devient autorisée et qu'un ou plusieurs bateaux attendent :

1. Le programme bloque l'entrée des nouveaux véhicules sur le pont.
2. Les véhicules déjà engagés terminent leur traversée.
3. Le pont passe de l'état FERMÉ à l'état OUVERTURE, puis à l'état OUVERT.
4. Les bateaux traversent un par un dans l'ordre FIFO.
5. Lorsque la file des bateaux est vide, le pont passe à l'état FERMETURE, puis revient à l'état FERMÉ.
6. Un bateau ne peut commencer sa traversée que lorsque le pont est complètement ouvert.

5. Délai minimal entre deux ouvertures

Une nouvelle ouverture du pont ne peut être autorisée qu'une heure après la fermeture complète de l'ouverture précédente.

À la fin de chaque fermeture, le programme mémorise l'heure exacte de fermeture complète et calcule l'heure de la prochaine ouverture autorisée :

T_prochaine_ouverture = T_fermeture_complete + 60 minutes (pour votre simulation, mettez plutôt 1 minute)
Lorsque l'heure autorisée est atteinte, le programme vérifie la présence de bateaux en attente et déclenche une nouvelle ouverture si nécessaire (utiliser un Timer/TimerTask)

Remarque : pour ne pas attendre, il est souhaitable qu'un bateau connaît l'heure prévue à laquelle une nouvelle ouverture pourra être autorisée (cette contrainte est non prise en compte dans le programme)

6. Gestion des véhicules prioritaires

Les ambulances et les véhicules de pompiers constituent une catégorie de véhicules prioritaires.

Lorsqu'un véhicule prioritaire arrive sur le pont ou à son entrée, il bénéficie d'une priorité sur les autres véhicules (voitures, les camions et les convois exceptionnels).

Pour simplifier la simulation, une bande d'urgence est prévue pour permettre aux véhicules se trouvant sur le pont de s'arrêter pour permettre le passage du véhicule prioritaire.

Le fonctionnement est le suivant :

1. Le programme détecte l'arrivée du véhicule prioritaire.
2. Les véhicules interrompent leur progression et se placent sur la bande d'urgence.
3. Chaque véhicule conserve l'association logique avec son ou ses segments de reprise : un segment pour une voiture, deux pour un camion et quatre pour un convoi exceptionnel.
4. Le véhicule prioritaire traverse le pont dans un sens, sans gêner les véhicules de l'autre sens
5. Lorsque le véhicule prioritaire a complètement quitte le pont, les véhicules interrompus reprennent leur progression à partir de leur état précédent (avant l'arrêt).

Consignes techniques

- Le projet doit être fait par un binôme.
- Ajouter des commentaires pour les différentes classes et leurs méthodes de votre projet.
- Enregistrer un vidéo de démonstration de deux minutes au maximum
- Le livrable est un dossier ayant comme nom les deux noms de binôme séparés par underscore, et contenant :
    - Les fichiers sources (.java), avec un schéma UML des classes créées
    - Le fichier .jar (pour pouvoir exécuter votre projet sur une ligne de commande).
    - Un fichier texte expliquant la ou les parties que vous n'avez pas pu les faire.

Annexe :

Pour pouvoir faire des animations des véhicules, la plus facile est d'utiliser la console en ligne en déplaçant le curseur. En Java, il existe plusieurs façons de placer le curseur dans une console avec la méthode : System.out.print(). Le choix dépend du terminal utilisé et de ce qu'on souhaite faire : revenir au début d'une ligne, aller à la ligne suivante, effacer l'écran ou placer le curseur à une ligne et une colonne précises.

Les différentes options dont certaines sont des séquences ANSI

"\n" : Passe à la ligne suivante.

"\r" : Revient au début de la ligne courante.

"\b" : Recule d'une position (backspace).

"\033[nG" : Place le curseur à la colonne `n` (ANSI).

"\033[n; mH" ; Place le curseur à la ligne `n`, colonne `m` (ANSI).

"\033[H" : Place le curseur en haut à gauche.

"\033[2J" : Efface l'écran (ANSI).

"\033[K" : Efface du curseur jusqu'à la fin de la ligne.

"\033[2K" : Efface toute la ligne.

On peut aussi utiliser **System.out.flush()** pour forcer l'affichage des modifications.